

attribute wrangle で VEX を書く ー 何が何回動くのか

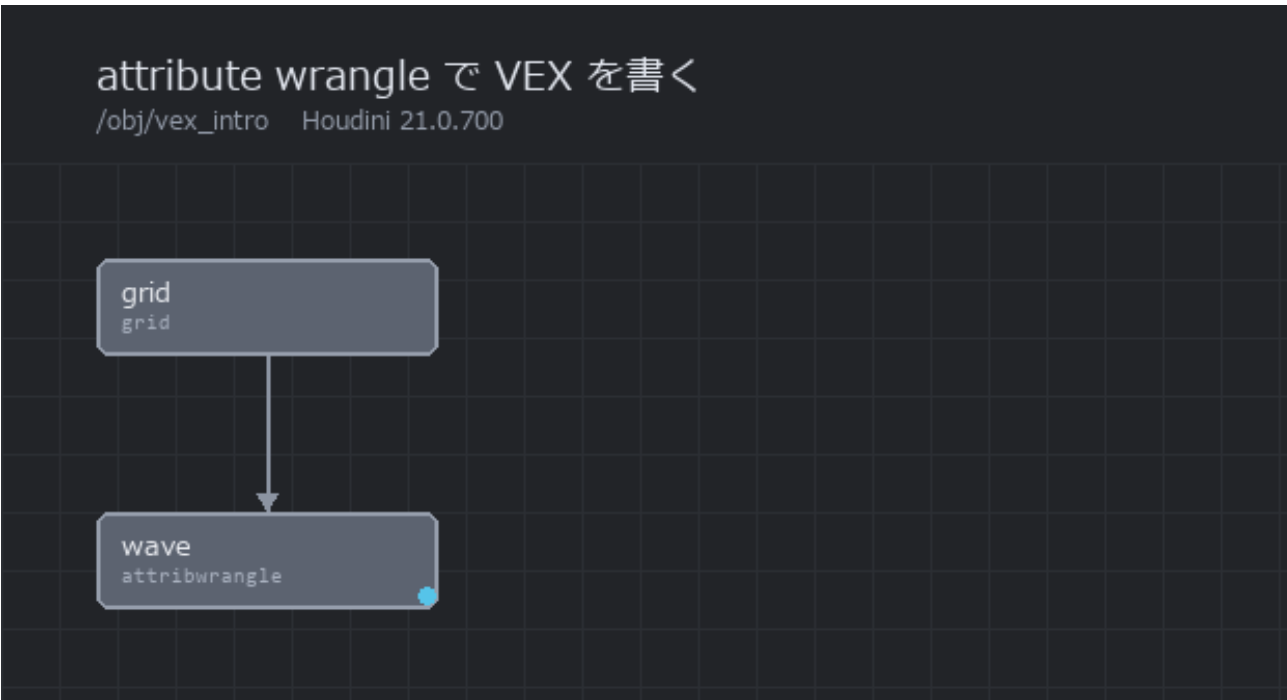
2026-09-13 | Houdini 21.0.700 | /obj/vex_intro | ノード 2 / 接続 1

何を作ったか

VEX は「ノードを並べる代わりに数行書く」ための言語。ただし初心者が最初につまずくのは文法ではなく、そのコードが何に対して何回走るのかという点。それを実測で確かめる。

あわせて「VEXは速い」という話を数字にしようとしたが、ここで測り方を二度も間違えた。最初の方法では計算量を100倍にしても時間が変わらず、公式の性能計測機能を使っても同じだった。三度目でようやく実際の計算を捉えられた。その経緯も残す。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 20 rows = 20
wave	attribwrangle	snippet = @P.y = sin(@P.x * 2.0) * 0.5;

実行対象を変えると、作られるアトリビュートの種類が変わる

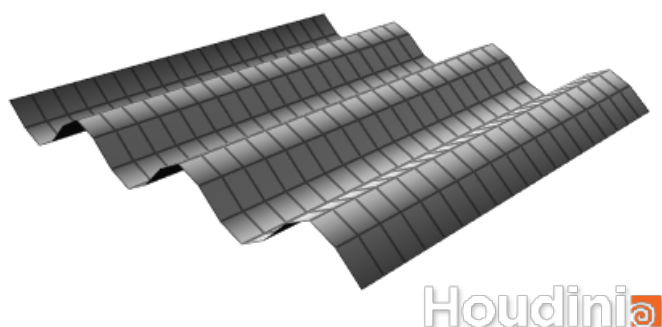
20×20 の grid（ポイント400 / プリミティブ361 / バーテックス1444）に対して、同じコード `@marker = 1.0;` を実行対象だけ変えて走らせた。

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Detail (only once)	ディテール	1
Primitives	プリミティブ	361
Points (既定)	ポイント	400

実行対象 (class)	作られたアトリビュートの種類	対象の要素数
Vertices	バーテックス	1444
Numbers	(作られない)	—

予測してから測る — $@P.y = \sin(@P.x * 2.0) * 0.5$

grid は 10×10 で原点中心なので x は -5 ~ +5。sin の値は -1 ~ +1 の範囲を取るので、y はちょうど -0.5 ~ +0.5 に収まるはず。



VEX 1行で作った波 $@P.y = \sin(@P.x * 2.0) * 0.5;$

	値
予測した範囲	-0.5 ~ +0.5
実測した範囲	-0.499983 ~ +0.499983
差	0.000017

点数と処理時間（毎回新しいノードを作り、5回測った中央値）

1秒あたりの点数が点数とともに増えているのは、ノードを作る固定の費用が大きいため。点が増えるほどその費用が薄まる。

分割数	点数	中央値（ミリ秒）	1秒あたりの点数
200	40,000	1.487	2,690万
400	160,000	2.005	7,979万
800	640,000	2.472	2億5,891万
1600	2,560,000	4.580	5億5,895万

その計測は本物か — 同じ点数で計算量だけ増やす（64万点）

計算量を約100倍にしたときに時間が増えなければ、計測が実際の計算を捉えていないということ。結果の最大yが変わっていることも、確かに別の計算が走った証拠になる。

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
軽い (sin × cos を1回)	2.48	1.0倍	0.49999

処理	中央値（ミリ秒）	軽いものとの比	結果の最大 y
重い（100回の繰り返し）	43.91	17.7倍	0.32169

わかったこと

- ・ attribute wrangle の「実行対象（class）」は5種類ある。Detail (only once) / Primitives / Points / Vertices / Numbers で、既定は Points。
- ・ 実行対象を変えると、作られるアトリビュートの種類がそれに従って変わる。同じ @marker = 1.0; というコードでも、Points なら点に、Primitives なら面に、Vertices なら頂点に、Detail ならジオメトリ全体に1つだけ作られる。コードは同じでも結果の置き場所が変わるというのが、最初に押さえるべき点。
- ・ Numbers だけはアトリビュートが作られなかった。これはジオメトリの要素ではなく、指定した個数の番号に対して走るモードなので、値を書き込む先がないため。
- ・ 予測を立ててから測る形にした。sin の値域は -1 ~ +1 なので、0.5 倍すれば y は -0.5 ~ +0.5 に収まるはず。実測は -0.499983 ~ +0.499983 だった。
- ・ この 0.000017 のずれも説明できる。grid は20分割なので点の間隔は $10/19 = 0.526316$ 。sin(2x) が最大になるのは $x = \pi/4 \approx 0.785398$ のときだが、最も近い格子点は $x = 0.789474$ で、わずかに外れている。この点での値を計算すると 0.4999834 になり、実測値と小数点以下6桁まで一致する。誤差ではなく、格子がピークをまたいでいるだけだった。
- ・ rand(@ptnum) は完全に再現する。同じコードを別のノードで2回走らせて、全400点が小数点以下9桁まで一致した。乱数といっても点番号から決まる固定の値なので、実行するたびに変わることはない。
- ・ その分布も妥当だった。0~2 の範囲に散らす設定で平均 1.03046（理論値 1.0）、範囲は 0.00103 ~ 1.99050。400点しかないので 3% ほどのずれは想定内。
- ・ 処理時間の計測は二度失敗した。一度目は cook(force=True) の前後を time.perf_counter で挟む方法。これだと計算量を100倍にしても時間が 0.9 倍にしかならなかった。ジオメトリを要求した時点で初めて計算されるため、計測区間の外で処理が走っていた。
- ・ 二度目は Houdini の性能計測機能（hou.perfMon）を使ったが、軽い処理も重い処理も同じ 0.008000 秒を返した。同じノードを焼き直す限り、キャッシュが効いて実際には再計算されないらしい。
- ・ 三度目にようやく成功した。毎回新しいノードを作って初回のクックを測る。キャッシュが存在しないので必ず計算が走る。この方法だと計算量100倍の処理が 17.7倍 の時間になり、結果の最大yも 0.49999 から 0.32169 に変わった。時間と結果の両方が動いたので、今度は本物。
- ・ 得られた数字。64万点に sin × cos を1回かける処理が 2.47ミリ秒、256万点で 4.58ミリ秒。この2点から傾きを出すと、1点あたりの限界コストは約1.1ナノ秒（毎秒約9億点）、固定の費用が約1.8ミリ秒。
- ・ つまり「VEXが速い」効果は点数が多いときにしか出ない。4万点では1.49ミリ秒かかっており、そのほとんどが固定費。少ない点数なら、どんな書き方をしても体感は変わらない。
- ・ 17.7倍という比率が100倍にならないのは、繰り返しの中身が単純で、ノードを作る固定費も含まれているため。それでも計算量に応じて時間が動くことは確認できたので、計測としては成立している。

次にやること

- ・ for-each ループと VEX の使い分け。同じことをどちらでもできる場面で、書きやすさと速さを比べる
- ・ VEX からアトリビュートを読み書きする（point() / setpointattrib()）
- ・ 測り方の失敗が3回続いたので、計測の型を作っておく。「結果が変わること」と「時間が変わること」の両方を確認するのを標準にする